

Formatadores personalizados na API Web ASP.NET

Core

O MVC do ASP.NET Core é compatível com a troca de dados em APIs web usando formatadores de entrada e saída. Os formatadores de entrada são usados pelo [Model Binding](#). Os formatadores de saída são usados para [formatar respostas](#).

A estrutura fornece formatadores de entrada e saída integrados para JSON e XML.

Fornece um formatador de saída integrado para texto sem formatação, mas não fornece um formatador de entrada para texto sem formatação.

Este artigo mostra como adicionar suporte para formatos adicionais criando formatadores personalizados. Para obter um exemplo de um formatador de entrada de texto sem formatação personalizado, confira [TextPlainInputFormatter](#) no GitHub.

[Exibir ou baixar código de exemplo \(como baixar\)](#)

Quando usar um formatador personalizado

Use um formatador personalizado para adicionar o suporte a um tipo de conteúdo que não é manuseado pelos formatadores integrados.

Visão geral de como usar um formatador personalizado

Para criar um formatador personalizado:

- Para serializar dados enviados ao cliente, crie uma classe de formatador de saída.
- Para desserializar os dados recebidos do cliente, crie uma classe de formatador de entrada.
- Adicione instâncias de classes de formatador às coleções de [InputFormatters](#) e [OutputFormatters](#) no [MvcOptions](#).

Criar um formatador personalizado

Para criar um formatador:

- Derive a classe da classe base apropriada. A amostra de aplicativo deriva de [TextOutputFormatter](#) e [TextInputFormatter](#).
- Especifique tipos de mídia e codificações que tenham suporte no construtor.
- Substitua os métodos [CanReadType](#) e [CanWriteType](#).

- Substitua os métodos [ReadRequestBodyAsync](#) e [WriteResponseBodyAsync](#).

O código a seguir mostra a classe do `VcardOutputFormatter` na [amostra](#):

C#

```
public class VcardOutputFormatter : TextOutputFormatter
{
    public VcardOutputFormatter()
    {
        SupportedMediaTypes.Add(MediaTypeHeaderValue.Parse("text/vcard"));

        SupportedEncodings.Add(Encoding.UTF8);
        SupportedEncodings.Add(Encoding.Unicode);
    }

    protected override bool CanWriteType(Type? type)
        => typeof(Contact).IsAssignableFrom(type)
        ||
        typeof(IEnumerable<Contact>).IsAssignableFrom(type);
}
```

Derivar da classe base apropriada

Para tipos de mídia de texto (por exemplo, vCard), derive das classes básicas [TextInputFormatter](#) ou [TextOutputFormatter](#):

C#

```
public class VcardOutputFormatter : TextOutputFormatter
```

Para tipos binários, derive das classes básicas [InputFormatter](#) ou [OutputFormatter](#).

Especificar codificações e tipos de mídia com suporte

No construtor, especifique as codificações e tipos de mídia com suporte ao adicioná-los às coleções de [SupportedMediaTypes](#) e [SupportedEncodings](#):

C#

```

public VcardOutputFormatter()
{

    SupportedMediaTypes.Add(MediaTypeHeaderValue.Parse("text/vcard
"));

    SupportedEncodings.Add(Encoding.UTF8);
    SupportedEncodings.Add(Encoding.Unicode);
}

```

Uma classe de formatador **não** pode usar injeção do construtor para suas dependências. Por exemplo, `ILogger<VcardOutputFormatter>` não pode ser adicionado ao construtor como um parâmetro. Para acessar serviços, você precisa usar o objeto de contexto que é transmitido para os métodos. Um exemplo de código neste artigo e a [amostra](#) mostram como fazer isso.

Substituir CanReadType e CanWriteType

Especifique o tipo no qual desserializar ou do qual serializar substituindo os métodos de [CanReadType](#) ou [CanWriteType](#). Por exemplo, criar um texto de vCard de um tipo de `Contact` e vice-versa:

C#

```

protected override bool CanWriteType(Type? type)
    => typeof(Contact).IsAssignableFrom(type)
    ||
    typeof(IEnumerable<Contact>).IsAssignableFrom(type);

```

O método CanWriteResult

Em alguns cenários, [CanWriteResult](#) deve ser substituído em vez de [CanWriteType](#). Use `CanWriteResult` se as condições a seguir forem verdadeiras:

- O método de ação retorna uma classe de modelo.
- Existem classes derivadas que podem ser retornadas no runtime.
- A classe derivada retornada pela ação deve ser conhecida no runtime.

Por exemplo, suponha que o método de ação:

- Assinatura retorna um tipo `Person`.
- Pode retornar um tipo `Student` ou `Instructor` que deriva de `Person`.

Se você quiser que o formatador manipule apenas objetos de `Student`, verifique o tipo de `Object` no objeto de contexto fornecido ao método `CanWriteResult`. Quando o método de ação retorna `ActionResult`:

- Não é necessário usar `CanWriteResult`.
- O método `CanWriteType` recebe o tipo de runtime.

Substituir `ReadRequestBodyAsync` e `WriteResponseBodyAsync`

A desserialização ou serialização é executada em `ReadRequestBodyAsync` ou `WriteResponseBodyAsync`. O exemplo a seguir mostra como obter serviços do contêiner de injeção de dependência. Os serviços não podem ser obtidos dos parâmetros de construtor:

C#

```
public override async Task WriteResponseBodyAsync(
    OutputFormatterWriteContext context, Encoding
    selectedEncoding)
{
    var httpContext = context.HttpContext;
    var serviceProvider = httpContext.RequestServices;

    var logger =
        serviceProvider.GetRequiredService<ILogger<VcardOutputFormatte
        r>>();
    var buffer = new StringBuilder();

    if (context.Object is IEnumerable<Contact> contacts)
    {
        foreach (var contact in contacts)
        {
            FormatVcard(buffer, contact, logger);
        }
    }
}
```

Configurar o MVC para usar um formatador personalizado

Para usar um formatador personalizado, adicione uma instância da classe do formatador à coleção

de `MvcOptions.InputFormatters` ou `MvcOptions.OutputFormatters`:

C#

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(options =>
{
    options.InputFormatters.Insert(0, new
VcardInputFormatter());
    options.OutputFormatters.Insert(0, new
VcardOutputFormatter());
});
```

Os formadores são avaliados na ordem em que são inseridos, sendo que o primeiro tem precedência.

A classe de `VcardInputFormatter` completa

O código a seguir mostra a classe do `VcardInputFormatter` na [amostra](#):

C#

```
public class VcardInputFormatter : TextInputFormatter
{
    public VcardInputFormatter()
    {
        SupportedMediaTypes.Add(MediaTypeHeaderValue.Parse("text/vcard"));

        SupportedEncodings.Add(Encoding.UTF8);
        SupportedEncodings.Add(Encoding.Unicode);
    }

    protected override bool CanReadType(Type type)
    => type == typeof(Contact);

    public override async Task<InputFormatterResult>
ReadRequestBodyAsync(
```

Testar o aplicativo

Execute a amostra de aplicativo para este artigo, que implementa formatadores básicos de entrada e saída de vCard. O aplicativo lê e grava vCards semelhantes ao seguinte formato:

```
BEGIN:VCARD
VERSION:2.1
N😊avolio;Nancy
FN:Nancy Davolio
END:VCARD
```

Para ver a saída do vCard, execute o aplicativo e envie uma solicitação Get com o `text/vcard` do cabeçalho Accept para `https://localhost:<port>/api/contacts`.

Para adicionar um vCard à coleção de contatos na memória:

- Enviar uma solicitação `Post` para `/api/contacts` com uma ferramenta como [http-repl](#).
- Defina o cabeçalho `Content-Type` como `text/vcard`.
- Configure um texto de `vCard` no corpo, formatado como o do exemplo anterior.